

smart-account-architecture

UnifyVault Account Abstraction & Self-Managed Gas Infrastructure Architecture (Phase 2A.5)

1. Executive Summary

Phase 2A.5 establishes a ****100% self-managed, provider-agnostic ERC-4337 Account Abstraction infrastructure**** for UnifyVault. All hard dependencies on third-party SaaS vendors (such as Pimlico billing, API keys, or credit-card requirements) have been completely removed.

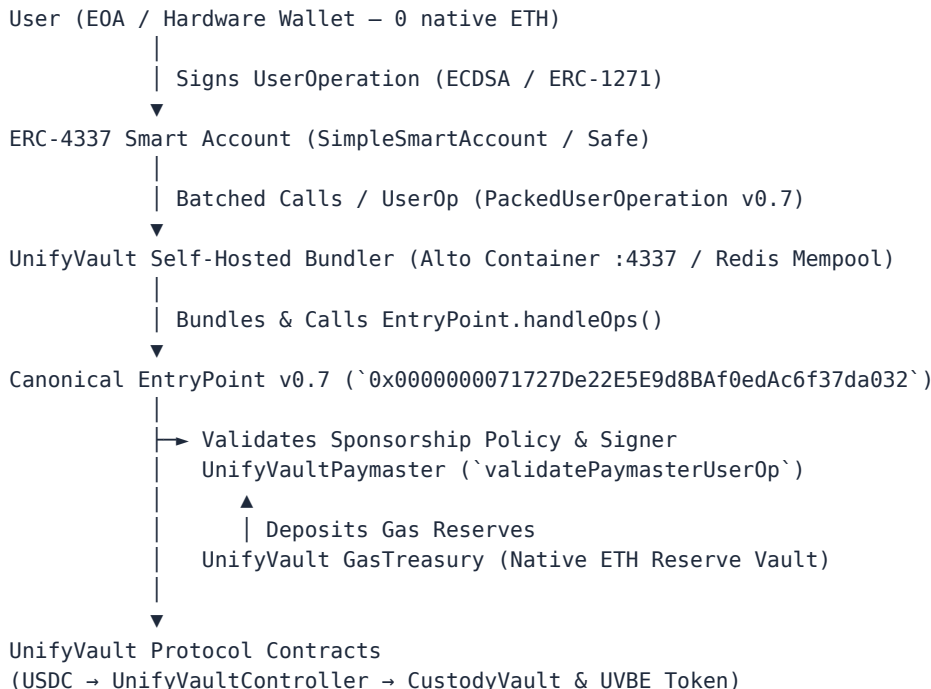
Core Economic Principle

****“Gasless” means the USER does not pay native ETH gas for protocol transactions.****

It does ****NOT**** mean the transaction has zero economic cost on the blockchain.

UnifyVault pays the underlying L1/L2 gas fees through its dedicated ****Gas Treasury**** and ****UnifyVaultPaymaster**** deposit on the canonical EntryPoint.

2. End-to-End System Architecture



3. Core Principles & Invariants

1. 100% Non-Custodial & EOA-Controlled

- The user's connected wallet (MetaMask, Coinbase Wallet, SafePal, Rainbow, etc.) remains the sole signing authority (`owner`).
- No user private keys are ever handled, generated, or stored by UnifyVault infrastructure.
- Smart Account addresses are derived deterministically counterfactual via the standard ERC-4337 factory on EntryPoint v0.7.

2. Core Protocol Invariance (Untouched Contracts)

The core UnifyVault protocol contracts remain ****100% unmodified****:

- `UnifyVaultController.sol`
- `UVBEV2.sol`
- `CustodyVault.sol`
- `CostBasisManagerV2.sol`
- `P2PEscrowV2.sol`

The Smart Account acts as the standard `msg.sender` for deposits, redemptions, transfers, and escrow trades.

3. Strict Accounting Isolation

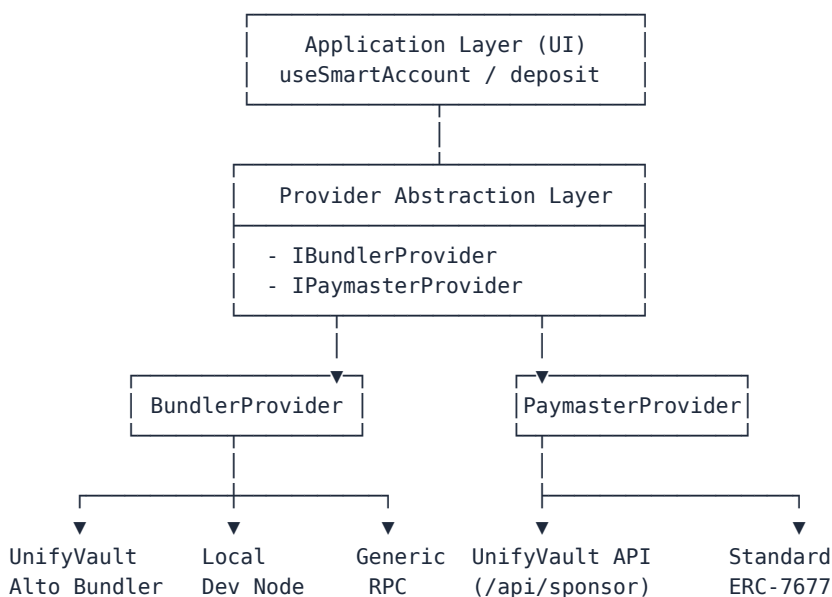
****Gas sponsorship is an external infrastructure operating expense.****

Paymaster gas funding is strictly isolated from all protocol asset accounting:

- Gas funding ****NEVER**** comes from `CustodyVault` collateral.
- Gas funding ****NEVER**** touches investor assets or escrow balances.
- Gas funding ****NEVER**** mutates user cost basis (`CostBasisManagerV2`).
- Gas funding ****NEVER**** affects NAV or share price calculations.
- Gas funding ****NEVER**** creates or distorts realized or unrealized P&L.

4. Provider Abstraction Layer

The frontend and backend interact with Account Abstraction through provider-agnostic interfaces:



Components:

1. `IBundlerProvider``: Standard interface for submitting `UserOperations` (`eth_sendUserOperation``), estimating gas (`eth_estimateUserOperationGas``), fetching receipts, and retrieving `UserOp` gas prices.
2. `IPaymasterProvider``: Interface for client-side pre-flight policy validation, generating ERC-4337 v0.7 stub data, and retrieving sponsored paymaster data.

3. `config.ts`: Configures `BUNDLER_RPC_URL`, `PAYMASTER_RPC_URL`, `PAYMASTER_ADDRESS`, and `AA_SPONSORSHIP_ENABLED` without requiring vendor API keys.
-

5. UnifyVault Paymaster Architecture (`UnifyVaultPaymaster.sol`)

The `UnifyVaultPaymaster` contract enforces strict on-chain validation for all sponsored `UserOperations`:

1. Canonical EntryPoint Binding

- Immutable `entryPoint` bound to canonical v0.7 (`0x0000000071727De22E5E9d8BAf0edAc6f37da032`).
- Only `EntryPoint` can invoke `validatePaymasterUserOp` and `postOp`.

2. Strict Calldata Inspection & Whitelists

- Target Whitelist**: Only approved contracts (`USDC`, `UnifyVaultController`, `UVBEV2`, `P2PEscrowV2`).
- Selector Whitelist**:
 - `USDC`: `approve(address,uint256)`
 - `Controller`: `deposit(address,uint256,uint256,address)`, `redeem(address,uint256,uint256,address,uint256)`
 - `UVBE`: `transfer(address,uint256)`, `approve(address,uint256)`
 - `P2PEscrow`: `createTrade`, `fundTrade`, `submitPaymentEvidence`, `confirmAndRelease`, `requestRefund`, `resolveDispute`
- Exact Allowance Enforcement**: In batched deposit operations (`approve` + `deposit`), the approved amount must strictly match the deposit amount. Leftover unspent allowance is forbidden.
- Zero Native Value**: Any call with `value > 0` is immediately rejected on-chain, preventing native ETH draining.

3. Anti-Drain & Anti-Spam Controls

- `maxCostPerUserOp`: Upper gas cost limit per `UserOperation` (default 0.05 ETH).
- `maxFeePerGasCap`: Maximum gas price cap (e.g. 100 gwei) preventing spikes from draining the treasury.
- `userOpCooldown`: Per-sender cooldown interval to prevent rapid automated spam.
- `isPaused`: Emergency pause switch controlled by governance/owner.

4. EntryPoint Gas Deposit Management

- `deposit()`: Deposits native ETH directly to the Paymaster's balance in `EntryPoint v0.7`.
 - `withdrawTo()`: Governance can withdraw unused gas funds to a designated cold wallet.
-

6. Gas Treasury Architecture (`GasTreasury.sol`)

The `GasTreasury` is a dedicated infrastructure reserve vault:

- Separation**: Distinct from `Treasury.sol` (protocol fee revenue) and `CustodyVault.sol` (collateral).
 - Automated Refills**: Designated `refillOperator` (relayer bot) can top up `UnifyVaultPaymaster`'s `EntryPoint` deposit when the balance drops below threshold.
 - Circuit Breakers**: Enforces `maxRefillPerTx` (e.g. 0.5 ETH) and `dailyRefillLimit` (e.g. 2.0 ETH) to cap exposure in case of relayer key compromise.
 - Admin Controls**: Governance multi-sig can withdraw funds, change limits, or pause during emergencies.
-

7. Self-Hosted Bundler Infrastructure (Alto OSS)

UnifyVault utilizes **Alto**, the production-grade, MIT-licensed open-source TypeScript/Node.js ERC-4337 v0.7 bundler.

VPS Deployment Specifications (`infra/bundler`):

- Runtime**: Docker Compose (`pimlicolabs/alto:latest` + `redis:7-alpine`).
- Resource Footprint**: 1-2 vCPUs, 2 GB RAM, lightweight storage. (Fits easily within UnifyVault's 4 vCPU, 16 GB RAM VPS).

- **RPC Requirement**: Standard Base Sepolia JSON-RPC node.
 - **Mempool**: Backed by Redis with strict replacement rules and rate limits.
 - **Observability**: Prometheus metrics exported on port `9090`.
-

8. Local Development Mode

In local development and test environments:

1. No external API keys or credit cards are required.
 2. If a live Bundler is not running, `BundlerProvider` and `PaymasterProvider` automatically provide safe simulated responses for development and unit testing.
 3. `/api/smart-account/sponsor` evaluates local policy and generates valid standard ERC-4337 v0.7 stubs.
-

9. Security Model & Threat Analysis

- **Threat Vector**: Mitigation Strategy
 - **Paymaster Gas Drain Attack**: Strict calldata inspection rejects arbitrary targets and non-zero ETH transfers; `maxCostPerUserOp` and `maxFeePerGasCap` enforce hard gas expenditure limits.
 - **Excessive Allowance Exploit**: Exact allowance matching ensures `USDC.approve` amount exactly matches `deposit` amount.
 - **UserOperation Replay**: EntryPoint v0.7 enforces sequential nonces and chain ID validation in userOp hashing.
 - **EntryPoint Spoofing**: Paymaster strictly checks `msg.sender == address(entryPoint)`.
 - **Mempool Spam / DoS**: Redis mempool limits pending userOps per sender; Bundler requires fee markup for replacements; Paymaster enforces sender cooldowns.
 - **Relayer Key Compromise**: GasTreasury enforces strict `maxRefillPerTx` and 24h rolling `dailyRefill-Limit`.
 - **Underfunded Paymaster**: Automated health checks monitor EntryPoint deposit balance and trigger alerts before gas exhaustion.
 - **Emergency Shutdown**: `UnifyVaultPaymaster` and `GasTreasury` both provide immediate owner `pause()` switches.
-

10. Failure Recovery Runbook

1. **Underfunded Paymaster**:
 - Alert fires when EntryPoint deposit < 0.05 ETH.
 - Relayer calls `gasTreasury.refillPaymaster(0.2 ether)`.
 - Manual fallback: Owner deposits native ETH directly via `paymaster.deposit{value: ...}()`.
1. **Bundler Crash or RPC Downtime**:
 - Docker container auto-restarts (`restart: unless-stopped`).
 - Web application falls back to secondary bundler RPC or standard EOA mode gracefully.
1. **Emergency Exploit Detected**:
 - Governance calls `paymaster.setPaused(true)` and `gasTreasury.setPaused(true)`.
 - All subsequent UserOperations revert in the validation phase without consuming paymaster gas.